

ChangeGuard Brief -- CG-101

Add Container Hold Status

Status: [!] Prerequisites + coordination required

Heads up

You are implementing a new **HOLD** lifecycle status, which introduces an "interrupt" state into the existing booking flow. This requires both API-level modifications to block transitions and frontend updates to visualize the new status.

Before you start

- **CG-105 (Refactor Booking Cutoff Logic):** This is a mandatory prerequisite. CG-105 moves the **LIFECYCLE** definitions and **checkCutoff** logic into a shared policy module. **Do not modify the LIFECYCLE array in bookings.ts until CG-105 is merged**, otherwise, you will face significant merge conflicts and likely break the new architecture.

Where you'll probably be working

- `server/src/routes/bookings.ts`: Core lifecycle array, status transition logic, and the new `/hold` and `/release-hold` endpoints.
- `client/src/components/StatusBadge.tsx`: Update **STATUS_STYLES** to include **HOLD** with a distinct color.
- `client/src/pages/BookingDetailPage.tsx`: Integration of the Hold/Release actions and status display.
- `docs/`: Update `data-model.md` and `api-reference.md`.

Tip: Treat these as starting points, not a fixed implementation plan. Verify the current code before making changes.

Watch for collisions

- **CG-103 (Audit Events):** This ticket will also be modifying the `bookings.ts` route handlers to log status transitions. Coordinate with the assignee to ensure your `/hold` and `/release-hold` endpoints trigger their audit event hooks correctly.
- **CG-102 (Block Billing Actions):** CG-102 depends on your work. They need to know when a booking is in **HOLD**. Ensure your **HOLD** status is properly exposed in the API response objects so their billing check logic can consume it.
- **CG-104 (CSV Export):** This is a minor collision in `BookingsListPage.tsx`. Since both tickets are UI-focused, check if they are modifying the same table rendering logic to avoid visual regressions or git conflicts.

Downstream impact

Tickets **CG-102** and **CG-103** rely on this implementation. Ensure that any status transitions or state helpers you add are robust and clearly documented, as these teams will be building their logic directly on top of your new status flag.

What this means for you

First, confirm with the owner of **CG-105** that their refactor is stable and merged into your target branch. Once done, implement the **HOLD** status within the new policy module introduced by CG-105. Prioritize the backend lifecycle logic first, as the downstream tickets (CG-102/103) depend on the new status being available and immutable.

Quick summary

Read first	CG-105
Coordinate with	CG-102, CG-103, CG-104
Highest collision risk	CG-105 (Architectural conflict)
Probably unrelated	None
Overall	[!] Prerequisites + coordination required